

Software Architecture Specification (SAS)

COS 301 - Capstone 2026
Capstone Project: EquityLens
By The Big Five



Project Name: EquityLens
Client: Amazon Web Services (AWS)
Team: The Big Five (TB5)

Team members:

- Abdulrahman Sabah (Team Lead) - u24566170
- Joshua Heath - u23541475
- Antony van Straten - u24590739
- Jonty Honey - u23536862

Contents

1. Introduction
2. Architectural Requirements and Architecture Diagram
3. Technology Requirements
4. API Contracts
5. Deployment
6. Non-functional requirements (NFR) Testing with evidence

1. Introduction

This document specifies how The Big Five (TB5) aimed to achieve the given specification provided by our client, Amazon Web Services.

2. Architectural Requirements

2.1 Architectural Patterns

2.1.1 Three-Tier Architecture

EquityLens follows a three-tier architecture:

- Presentation tier - a single-page web application responsible for rendering the UI and handling user interaction. It holds no business logic and never accesses the database directly.
- Application tier - an API backend containing all business logic: authentication orchestration, portfolio persistence and live pricing, indicator calculation, news correlation, AI prompt construction and guardrails.
- Data tier - a relational database storing user accounts, portfolio data, snapshots, watchlists, conversations, and cached market/news data.

The tiers communicate exclusively over HTTPS REST. This separation exists so each tier can be developed, tested, scaled and replaced independently.

2.1.2 Serverless Architecture

The application tier targets deployment as serverless functions behind a managed API gateway. Functions execute on demand, scale automatically with traffic, and incur no cost when idle unless implemented otherwise.

2.1.3 Layered Backend

Within the application tier the backend is layered:

- Routers handle HTTP concerns only: request parsing, response shaping, status codes, auth dependency injection.
- Services encapsulate business logic: portfolio aggregation and live pricing, indicator maths, news correlation, AI context building and guardrails.
- Repositories encapsulate data access: each aggregate (portfolios, holdings, transactions, users, watchlist, market data) has a repository owning its queries.

This means business logic is unit-testable without HTTP, and queries are changeable without touching business logic.

2.1.4 Client-Server with External Service Integration

Market data, news, identity and AI capabilities are consumed from external providers through dedicated service modules, each wrapped with caching and/or graceful degradation behaviour so that a provider outage degrades its own feature

2.2 Design Patterns

- Service Layer - backend/app/services/. Separates business logic from HTTP routing, services are unit-tested independently of the API layer.

- Repository - backend/app/repositories/. Each aggregate's data access is isolated behind a repository (e.g. PortfolioRepository, HoldingsRepository, UserRepository), so query changes never leak into business logic.
- Provider - frontend/src/context/. React context providers (AuthContext, PortfolioContext, ThemeContext) expose global state without prop drilling.
- Observer - React state/context updates. Component re-renders are driven by state changes - an implicit observer model across the UI.
- Factory - get_db() in app/database.py and get_bedrock_client() in the AI service. Consistent construction and lifecycle management of database sessions and external service clients.
- Facade - usePortfolio, useAuth and related hooks. Each hook presents a single simple interface over multi-step fetch/auth flows for UI components.
- Cache-Aside - app/utils/stock_cache.py and market_cache.py. Price history and fundamentals are looked up in the object-store cache first and fetched from the provider only on a miss, respecting rate limits.

2.3 Constraints

2.3.1 Financial Constraints

- **C-1:** All infrastructure must operate within AWS Free Tier limits at zero cost.
- **C-2:** All non-AWS tools must be open-source or free-tier.
- **C-3:** AI capabilities must use free-tier LLM APIs exclusively.

2.3.2 Security and Compliance Constraints

- **C-4:** All data must be encrypted in transit (TLS) and at rest (AES-256).
- **C-5:** All user data handling must be fully compliant with POPIA.
- **C-6:** Raw brokerage credentials must never be stored in the system.
- **C-7:** Portfolio data may only be imported via file upload or manual entry.
- **C-8:** User portfolio data must not be visible to any other user.

2.3.3 Integration Constraints

- **C-9:** Market data fetching must respect free-tier API rate limits, with caching to avoid redundant calls.
- **C-10:** The platform must support at least one South African brokerage for portfolio statement import.

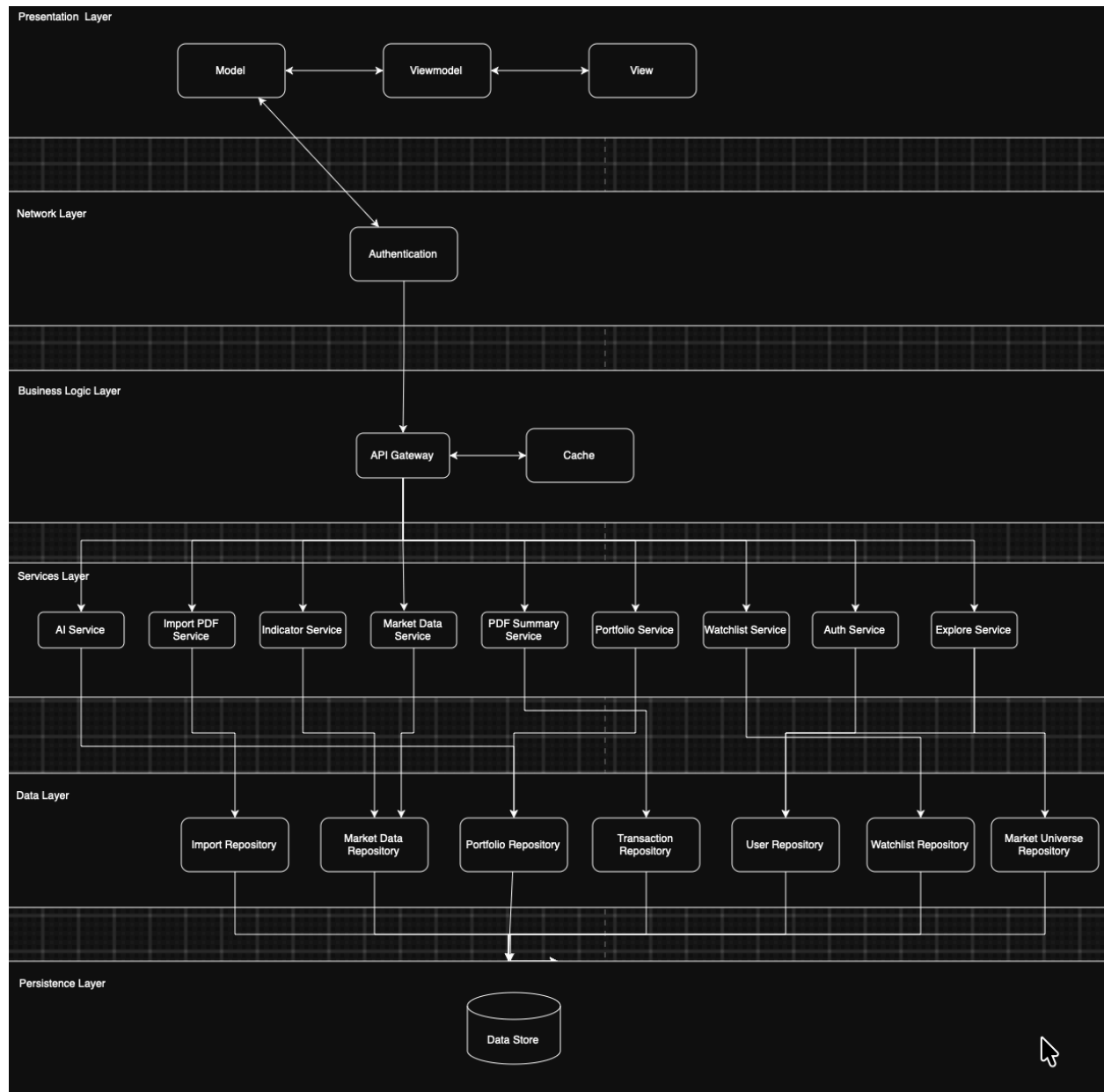
2.3.4 AI Constraints

- **C-11:** The conversational AI must include documented guardrails preventing explicit buy/sell recommendations.
- **C-12:** Guardrails must be implemented at the system-prompt level and documented.

2.3.5 Scalability Constraints

- **C-13:** The architecture must support multiple concurrent users and growing volumes of historical market data.
- **C-14:** Horizontal scaling must be achievable without significant architectural changes.

2.4 Architectural Diagram



2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Justification
Security - 100% of protected endpoints reject unauthenticated requests, MFA enforced, TLS + AES-256, strict per-user data scoping	Authentication delegated to a managed identity provider (AWS Cognito) providing email verification and TOTP MFA, every protected router resolves the user via a shared <code>get_current_user</code> dependency and repositories filter every query by the resolved user's ID, TLS termination at the managed gateway/CDN, managed database and object-store encryption at rest, no brokerage credentials ever collected
Performance – p85 API response ≤ 1000 milliseconds, dashboard renders ≤ 5 seconds	Cache-aside pattern for market data: an instrument already in the object-store cache is served with zero provider round-trips, a single aggregated dashboard endpoint (<code>GET /api/portfolio</code>) instead of N per-widget calls, indexed foreign keys on all portfolio-scoped tables, live pricing bounded to what the dashboard displays.
Scalability – The Big Five has no real evidence of this project performing at scale as of yet	Stateless API (session state lives with the client token and database), so the container can be redeployed or replaced without losing session state; relational schema with explicit indexes on all high-volume lookup paths (<code>`portfolio_id`</code> , <code>`ticker`</code> , <code>`user_id`</code>); historical data growth handled by the same indexed query paths.
Usability & Accessibility - first import to populated dashboard ≤ 5 min, WCAG 2.1 AA, responsive 320px+, reduced-motion support	Single-page application with client-side routing for instant page transitions, design-token-driven theming (light/dark) with contrast-checked variables, global keyboard focus-visible styling, empty states designed as onboarding paths (dashboard prompts import when no holdings exist), prefers-reduced-motion respected by animated components.

3. Technology Requirements

3.1 Frontend

- React + Vite (framework) - component model fits the dashboard's card/panel structure, Vite gives fast dev feedback, large ecosystem and team familiarity reduce delivery risk.
- TailwindCSS + CSS design tokens (styling) - utility classes keep styling co-located with components, CSS variables implement the brand style guide's tokens and light/dark themes.
- Recharts (data visualisation) - declarative React charting for the performance, sector and dividend charts without custom SVG work.
- SheetJS (xlsx) (spreadsheet parsing) - parses the Excel import template client-side before preview/confirm.
- Docker and nginx (hosting) - the production build (Dockerfile.prod) is served by nginx from a container on the same EC2 instance as the backend (Section 5.2).

3.2 Backend

- FastAPI (Python 3.11) (framework) - async-capable, automatic OpenAPI documentation (kept in docs/api/openapi.json), Pydantic validation enforcing the API contracts, Python gives direct access to the financial computation ecosystem.
- pandas + numpy (financial computation) - vectorised indicator calculations over price history.
- SQLAlchemy + Alembic (ORM and migrations) - versioned, reversible schema migrations, CI verifies upgrade to downgrade to upgrade on every run.
- Docker on a single AWS EC2 instance (compute) - the FastAPI app is built into a Docker image and run as a container directly on one EC2 host (docker run -p 8000:8000).

3.3 Data and External Services

- AWS RDS PostgreSQL (database) - relational integrity for the portfolio hierarchy (portfolios to holdings to transactions), encryption at rest, free-tier eligible, indexes satisfy.
- AWS Cognito (authentication) - managed user pool providing email verification codes and TOTP MFA without storing credentials ourselves, tokens verified server-side per request.
- AWS Bedrock (Anthropic Claude) (AI) - managed LLM inference inside the AWS estate, system-prompt guardrails, portfolio context assembled server-side so no third party receives raw user data beyond the inference call.
- yfinance (market data) - free access to JSE price history and fundamentals, covers prices, history and search. Resilience comes from the database cache plus the per-holding cost-basis fallback (Reliability) and Alpha Vantage API fallback for NFR-3.2 hardening.
- NewsData (news) - free-tier financial news APIs covering South African tickers, named in the SRS deliberately to keep the requirement unambiguous.

3.4 Delivery and Operations

- GitHub Actions (CI/CD) - free for the repository, runs the full quality gate on every push/PR.
- Docker + Docker Compose (containerised development) - one-command reproducible local environment (frontend, backend, database) so a fresh clone of main runs with only the repository's instructions.
- Amazon ECR (container registry) - stores the built frontend and backend Docker images, tagged by commit SHA plus a mutable latest. The deploy workflows pull from here onto the EC2 host.

Development tools: Git (CLI), GitHub (repo, issues, project board), Postman (endpoint testing), VS Code.

Testing frameworks: Vitest + React Testing Library (frontend unit/component), Playwright (E2E), pytest + pytest-cov (backend unit/integration), Codecov (coverage reporting), npm audit / pip-audit / Bandit / Trivy (dependency and security scanning in CI).

4. API Contracts

All endpoints are prefixed `/api` and return JSON. Unless marked (*no auth*), endpoints require a bearer access token in the Authorization header and return 401 when it is missing, expired or invalid. Full request/response schemas are auto-generated into `docs/api/openapi.json` from the backend's validation models.

4.1 Authentication - `/api/auth`

- **POST** `/api/auth/register` (*no auth*) - register with full name, email, password. 201 on success, 409 if the email is already registered, 422 on validation failure.
- **POST** `/api/auth/confirm` (*no auth*) - confirm the 6-digit email verification code. 400 on invalid/expired code.
- **POST** `/api/auth/login` (*no auth*) - authenticate email + password. Returns either tokens (`access_token`, `id_token`, `refresh_token`) or an MFA challenge (`challenge`, `session`) that must be completed. 401 on invalid credentials.
- **POST** `/api/auth/mfa/verify-login` (*challenge session*) - complete an MFA challenge with the 6-digit TOTP code, returns tokens. 401 on invalid code.
- **POST** `/api/auth/mfa/associate` - begin TOTP setup, returns the shared secret for QR display.
- **POST** `/api/auth/mfa/confirm-setup` - verify a TOTP code to activate MFA for the account.
- **GET** `/api/auth/me` - return the authenticated user's profile.
- **POST** `/api/auth/logout` - invalidate the user's sessions.

4.2 Portfolio - `/api/portfolio`

- **GET** `/api/portfolio` - aggregated dashboard payload: summary (total value, cost, gain/loss, daily change), live-priced holdings, sector allocation, performance history, dividend history. The daily snapshot is recorded on this call.
- **GET** `/api/portfolio/summary` - summary statistics only.
- **GET** `/api/portfolio/sectors` - sector allocation breakdown.
- **GET** `/api/portfolio/performance` - snapshot history (portfolio value + benchmark per day).

4.3 Portfolio Import - `/api/import_pdf` and `/api/import_pdf_summary`

Statement content is extracted client-side

- **POST** `/api/import_pdf/` - register an uploaded statement document, returns the document record for the save calls below.
- **POST** `/api/import_pdf/save_portfolios` - persist confirmed portfolio metadata.
- **POST** `/api/import_pdf/save_holdings` - persist confirmed holdings.
- **POST** `/api/import_pdf/save_instrument_purchases_and_sales` - persist confirmed buy/sell transactions.
- **POST** `/api/import_pdf/save_contributions_and_withdrawals` - persist confirmed cash movements.

- **POST** /api/import_pdf/save_dividends_and_withholding_tax - persist confirmed dividend records.
- **POST** /api/import_pdf/save_transaction_expenses - persist confirmed fee/expense records.
- **GET** /api/import_pdf_summary/summary/{portfolioID} - statement summary for an imported portfolio.
- **GET** /api/import_pdf_summary/top_holdings/{portfolioID} - largest holdings in the statement.
- **GET** /api/import_pdf_summary/lowest_holdings/{portfolioID} - smallest holdings in the statement.
- **GET** /api/import_pdf_summary/portfolio_allocation/{portfolioID} - allocation breakdown.
- **GET** /api/import_pdf_summary/trading_activity/{portfolioID} - buy/sell activity.
- **GET** /api/import_pdf_summary/cash_flow/{portfolioID} - contributions and withdrawals.
- **GET** /api/import_pdf_summary/dividend_income/{portfolioID} - dividend income.
- **GET** /api/import_pdf_summary/expenses/{portfolioID} - fees and expenses.

4.4 Market Data - /api/stocks

- **GET** /api/stocks/details?symbol= - current price, volume and daily change for a ticker, error response if no data is available.
- **GET** /api/stocks/history?symbol=&period= - historical OHLCV data, period accepts the standard ranges (1d, 5d, 1mo, 3mo, 6mo, 1y, 2y, 5y, 10y, ytd, max).
- **GET** /api/stocks/search?query= - search stocks by ticker or company name.

4.5 News - /api/news

- **GET** /api/news/all - latest general financial news.
- **GET** /api/news/?category= - news filtered by category, with holding-relevant naming applied server-side.

4.6 AI Assistant - /api/ai_chat

- **POST** /api/ai_chat/ - submit a message ({message, conversation_id?}). Builds portfolio context, applies guardrails, returns {reply, conversation_id}. Starting a conversation auto-generates a title.
- **GET** /api/ai_chat/conversations/ - list the user's conversations (id, title, timestamps), most recent first.
- **GET** /api/ai_chat/conversations/{id}/messages/ - full message history for a conversation.
- **PUT** /api/ai_chat/conversations/{id}/ - rename a conversation ({title}).
- **DELETE** /api/ai_chat/conversations/{id}/ - delete a conversation and its messages.

4.7 Watchlist - /api/watchlist

- **POST** /api/watchlist/ - add a ticker to the user's watchlist ({ticker}), enriched server-side with company name and sector.

- **GET** /api/watchlist/ - list the user's watchlist entries with current market data.
- **DELETE** /api/watchlist/{watchlistID} - remove an entry.

4.8 System

- **GET** /health (*no auth*) - health check, returns {"status": "ok"}. Exposed for external uptime monitoring.

4.9 Financial Indicators - /api/indicators

- **GET** /api/news/all - latest general financial news.
- **GET** /api/news/?category= - news filtered by category, with holding-relevant naming applied server-side.

5. Deployment

5.1 Deployment Requirements

Live, accessible system: the production system is reachable at [equitylens.co.za\]\(https://equitylens.co.za/\)](https://equitylens.co.za/)

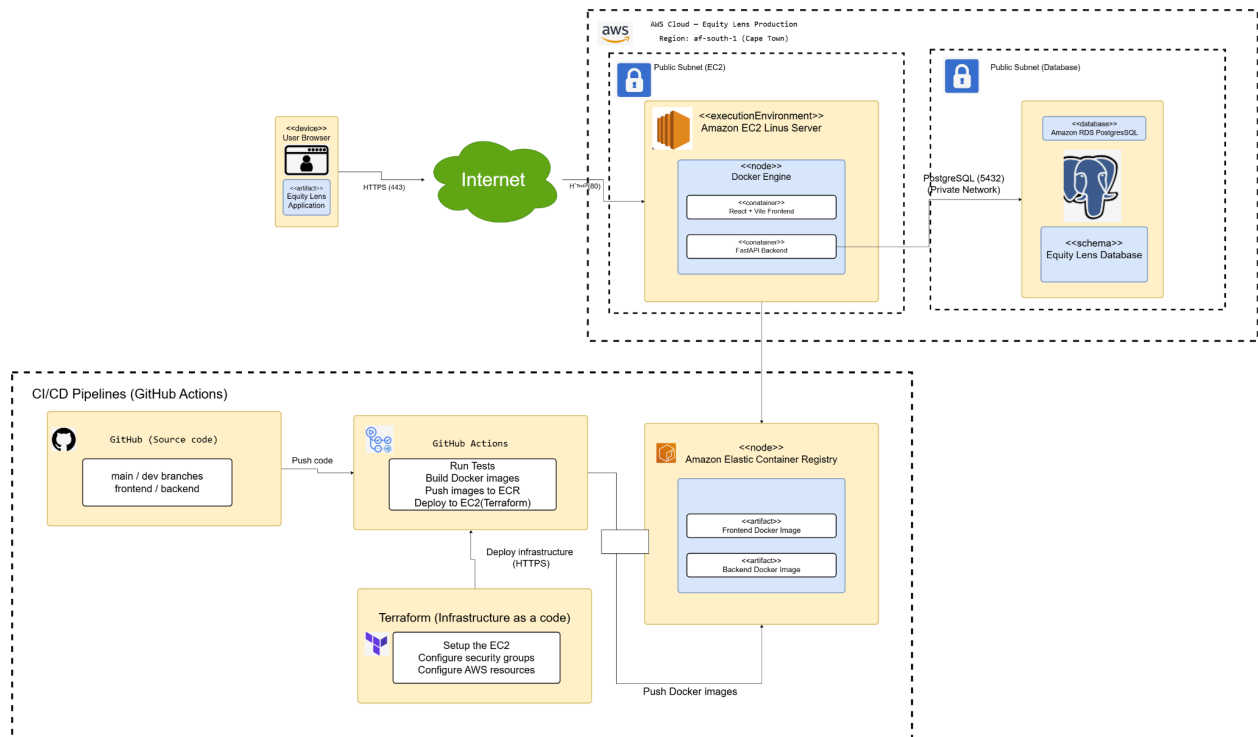
Environment parity: `deploy-backend.yml` and `deploy-frontend.yml` both trigger their AWS/deploy steps on `pull_request` events targeting `main` or `dev`, and both deploy to the same fixed container names on the same single EC2 instance. There is no separate staging AWS stack. A PR opened against `dev` deploys to the same production instance as a PR against `main`.

Development - Docker Compose on each developer's machine (local PostgreSQL, hot reload)

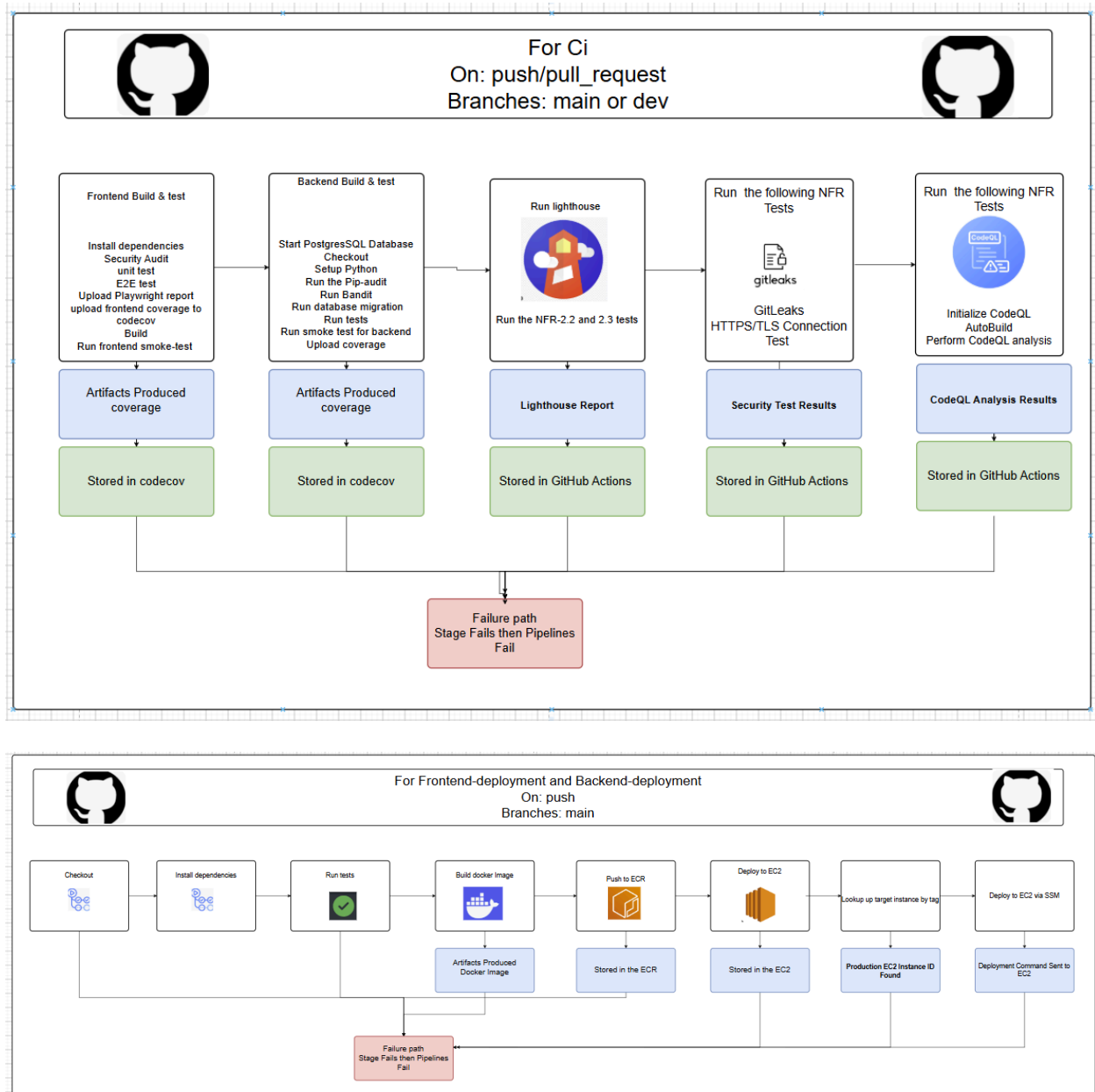
Production - the single EC2 instance, redeployed whenever a qualifying PR event fires

Reproducibility (containerisation) - local environments are fully containerised: a fresh clone of `main` runs with `docker compose up --build` using only the repository's instructions. Production is also containerised (Docker images pushed to Amazon ECR, pulled and run on the EC2 host)

5.2 Deployment Diagram



5.3 CI/CD Pipeline Diagram



5.4 Secrets Management

Sensitive information such as the API keys, and AWS credentials, will be stored secretly in the GitHub Actions (Secrets and environment variables), and we will also ensure that no sensitive information will be committed in any branch for security purposes. To see these variables, you can visit our GitHub in the secrets and environments variables, where you will see the following.

Repository secrets New repository secret

Name 	Last updated
 AWS_ROLE_ARN	3 weeks ago  
 CODECOV_TOKEN	4 months ago  
 EC2_PUBLIC_ADDRESS	2 months ago  
 EC2_USERNAME	2 months ago  
 TERRAFORM_APPLY_ROLE_ARN	3 weeks ago  
 TERRAFORM_PLAN_ROLE_ARN	3 weeks ago  
 THE_PRIVATE_SSH_KEY	2 months ago  

5.5 Rollback Strategy

- Backend: here is no immutable-version rollback anymore. The deploy script always pulls and runs the :latest ECR tag; rolling back today means manually SSHing to the EC2 host and running `docker pull <repo>:<previous-sha>` followed by the same `stop/rm/run` sequence the deploy script uses, since ECR does retain each commit's SHA-tagged image. There is no automatic rollback-on-failure: the old container is stopped and removed before the new one starts, so a crash-on-startup in the new image currently means downtime with nothing to fall back to automatically.
- Frontend: The frontend is a Docker/nginx container on the same EC2 instance. Rollback is a manual re-tag-and-redeploy procedure, similar to the one above.
- Database: Alembic migrations are written to be reversible and CI proves it on every run (upgrade to downgrade to upgrade), so a schema rollback is alembic downgrade to the previous revision. RDS automated snapshots cover data-level recovery.
- Process: main only receives merges from dev that passed the full pipeline plus two reviews, so the previous main commit is always a known-good deployable state, the default rollback action is revert the merge commit and let the pipeline redeploy.

6. Non-functional requirement (NFR) Testing

To confirm that the non-functional requirements specified in the SRS are met, our team (EquityLens) will employ both executable and repeatable tests. We will compare the actual outcome to the objective specified in our SRS document when each quality requirement is met utilizing a method or tool. Additionally, we have included a screenshot and test outputs that serve as evidence of whether each of the following requirements has been met.

Every NFR has its own tool, such as OWSAP or K6, and we conducted all testing using the GitHub action workflow. Once it has been pushed to either dev or main, the outcome may be viewed to determine whether it was successful or unsuccessful. Next, we took the screenshot that is displayed below. Some other test were run locally and screenshot were taken as evidence

Security Testing

ID	Executable tests/ procedures	Tool	Target/ expected results	Actual result/ evidence
NFR-1.1	Attempting to use every protected API without logging in.	OWSAP ZAP	100 % of protected routes return HTTP 401.	See evidence below
NFR-1.2	To check that all the passwords are hashed in the database, tokens expire within 60 minutes, and of course all expired tokens are rejected.	Live system test + AWS Cognito	The database will not have a plain text password; tokens that are older than sixty minutes will be discarded.	See evidence below
NFR-1.3	To try to log in with an invalid TOTP, then with a valid TOTP.	Live Login Test	The login will only be succeeded, if the TOTP is valid.	See evidence below
NFR-1.4	To check the websites use "HTTPS" and not "HTTP" and all the stored data is encrypted.	OpenSSL	The HTTP/TLS is used on the website.	See evidence below
NFR-1.5	All portfolio data shall be scoped to the owning user at the data-access level: 0 instances of cross-user data exposure.	OWSAP ZAP	No one should be able to access other people portfolio	See evidence below
NFR-1.6	Looks for any brokerage usernames, passwords, and API keys in the database.	GitLeaks	There are 0 raw brokerage credentials stored.	See evidence below

NFR-1.7	This checks that the website (EquityLens) follows all the rules of POPIA privacy requirement.	OWSAP ZAP	All the defined POPIA acceptance passes.	See evidence below
----------------	---	-----------	--	---------------------------

Evidence of the Security Testing

NFR-1.1 and 1.7 and 1.5

In here we run the OWSAP command locally in order to test it, and we only got warning errors

Command to run:

- docker compose up -d
- docker network ls
- docker run --rm -t `

--network equity-lens_default `

ghcr.io/zaproxy/zaproxy:stable `

zap-api-scan.py `

-t http://equity-lens-backend-1:8000/openapi.json `

-f openapi

```
PS C:\Users\Abdul\Documents\Equity-Lens> docker run --rm -t `
>> --network equity-lens_default `
>> ghcr.io/zaproxy/zaproxy:stable `
>> zap-api-scan.py `
>> -t http://equity-lens-backend-1:8000/openapi.json ` ...
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 3
    http://equity-lens-backend-1:8000/openapi.json (200 OK)
    http://equity-lens-backend-1:8000/health (200 OK)
    http://equity-lens-backend-1:8000/api/news/test-aapl/ticker (200 OK)
WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 3
    http://equity-lens-backend-1:8000/openapi.json (200 OK)
    http://equity-lens-backend-1:8000/health (200 OK)
    http://equity-lens-backend-1:8000/api/news/test-aapl/ticker (200 OK)
WARN-NEW: Application Error Disclosure [90022] x 1
    http://equity-lens-backend-1:8000/api/auth/mfa/verify-login (500 Internal Server Error)
FAIL-NEW: 0    FAIL-INPROG: 0    WARN-NEW: 4    WARN-INPROG: 0    INFO: 0    IGNORE: 0    PASS: 115
```

NFR-1.2

1). Password is hashed

Users [Info](#)

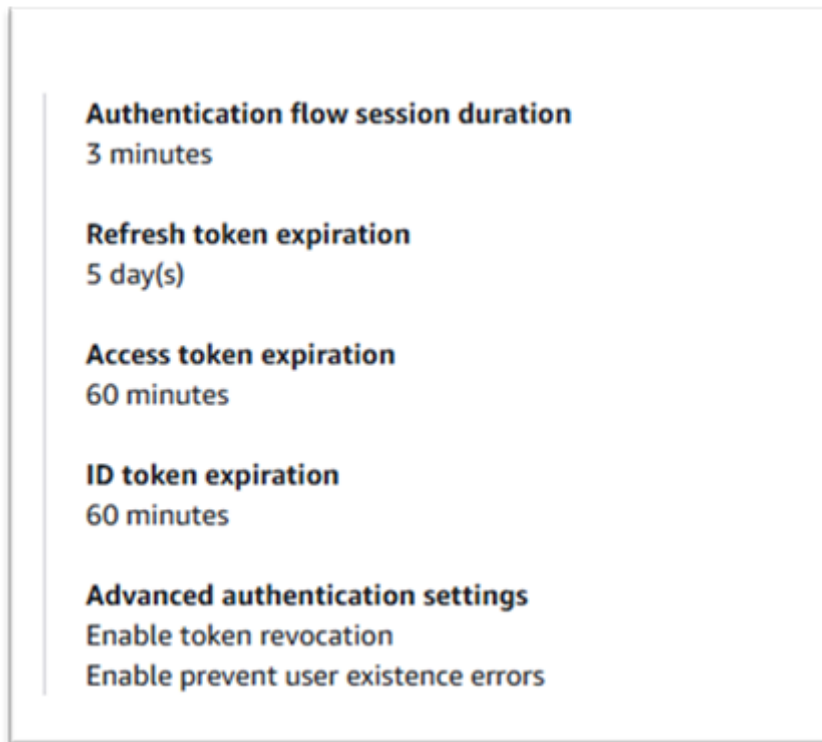
View, edit, and create users in your user pool. Users that are enabled and confirmed can sign in to your user pool.

Property: User name Search users by attribute

	User name	Email address	Email verified	Confirmation status	Status
☐	914c3248-e091-708d-ca34-69df14bf...	zaproxy@example.com	No	Unconfirmed	✓ Enabled
☐	41bc22b8-c051-70d9-343e-8e9f4eff6...	joshuaheath004@gmail.com	Yes	Confirmed	✓ Enabled
☐	c13c2288-a081-7088-b847-99a1d2b3...	wedsaq5@gmail.com	Yes	Confirmed	✓ Enabled
☐	e13c32b8-c061-70fd-bc3c-e1440e38f...	123@tuks.co.za	No	Unconfirmed	✓ Enabled
☐	111c3278-2061-7010-9407-a8f9b72a...	floshheath@gmail.com	Yes	Confirmed	✓ Enabled
☐	d1ace2e8-a071-702c-21b4-d301008f...	thefivebigtb5@gmail.com	No	Unconfirmed	✓ Enabled
☐	815ce248-3001-70ff-8c89-747f2bb07...	test@example.com	No	Unconfirmed	✓ Enabled
☐	41dc1268-d0c1-70d8-163b-d82b982...	officialiceyxd@gmail.com	Yes	Confirmed	✓ Enabled
☐	715c0278-e081-70bf-ba29-8c064fe69...	u24590739@tuks.co.za	Yes	Confirmed	✓ Enabled
☐	916c42c8-a0f1-7055-6f18-0b6cc39a4...	floshheath+t1@gmail.com	No	Unconfirmed	✓ Enabled

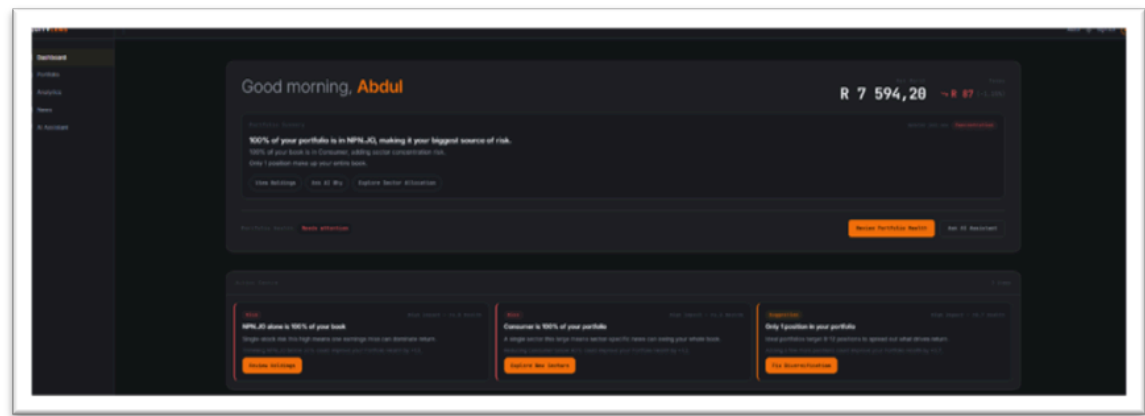
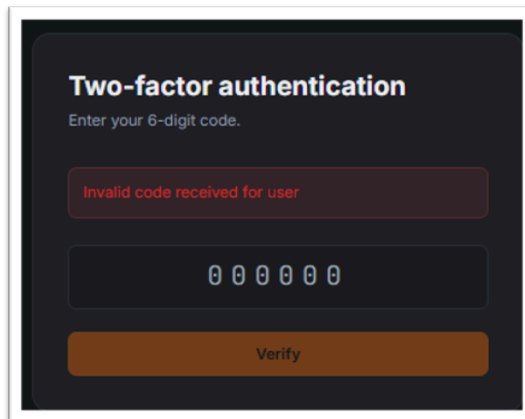
So, the database does not store the password, and the password is protected by Cognito which does not display the passwords in the AWS console for security reasons.

2). Token expiry date



As we can see from the screenshot that both the access token and the ID token are configured in AWS to expire after 30 minutes

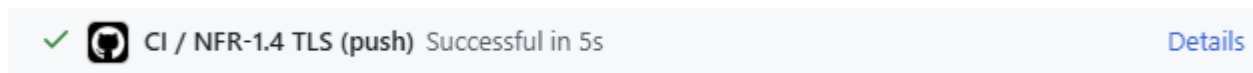
NFR-1.3



After logging in, he must verify his password using Google Authenticator. If it is incorrect, an error notice appears on the left, and if it is valid, I will be taken to the dashboard, which is visible on the right.

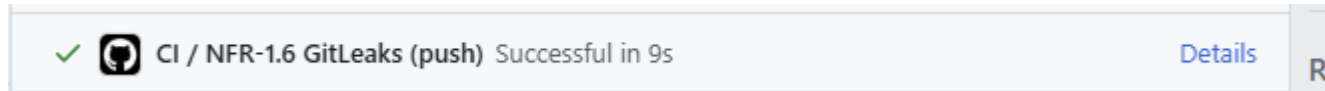
NFR 1.4

This test was conducted in the pipelines, and when it passed, a screenshot is displayed below.



NFR 1.6

This test was conducted in the pipelines, and when it passed, a screenshot is displayed below.



Performance Testing

ID	Executable tests/ procedures	Tool	Target/ expected results	Actual result/ evidence
NFR-2.1	sending the API request and timing the response.	K6	85% of requests will be answered in 500 milliseconds.	See evidence below
NFR-2.2	Measure how long it takes for the first page to load after opening the website (live).	Lighthouse	The page needs to load in 10 seconds or less.	See evidence below
NFR-2.3	To measure how long it takes for the charts and graphs to show after opening the dashboard page.	Lighthouse	All the holdings graphs and charts (dashboard visualizations) appear in 6 seconds.	See evidence below
NFR-2.4	To make two requests for the identical market data and verify that the second request makes use of the cache.	Live API Test	The second call makes 0 calls to the external provider.	See evidence below

Evidence of the Performance Testing

NFR-2.1

Command to run: k6 run backend/tests/k6/performance.js

```
terminal
PS C:\Users\Abdul\Documents\Equity-Lens> k6 run backend/tests/K6/performance.js

THRESHOLDS

http_req_duration
✓ 'p(85)<500' p(85)=75.04ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 25312 840.237302/s
checks_succeeded...: 100.00% 25312 out of 25312
checks_failed.....: 0.00% 0 out of 25312

✓ status is 200

HTTP
http_req_duration.....: avg=57.04ms min=22.3ms med=49.75ms max=590.72ms p(90)=84ms p(95)=98.14ms
{ expected_response:true }...: avg=57.04ms min=22.3ms med=49.75ms max=590.72ms p(90)=84ms p(95)=98.14ms
http_req_failed.....: 0.00% 0 out of 25312
http_reqs.....: 25312 840.237302/s

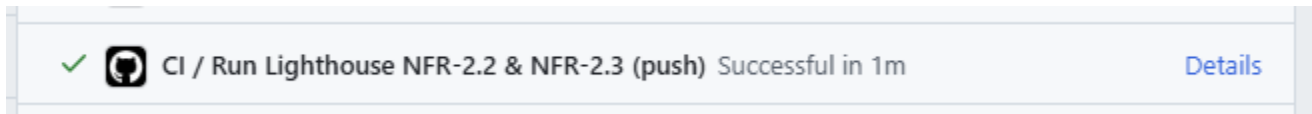
EXECUTION
iteration_duration.....: avg=59.29ms min=24.32ms med=50.04ms max=2.09s p(90)=86.22ms p(95)=101.42ms
iterations.....: 25312 840.237302/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 6.4 MB 213 kB/s
data_sent.....: 3.1 MB 103 kB/s

running (0m30.1s), 00/50 VUs, 25312 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
```

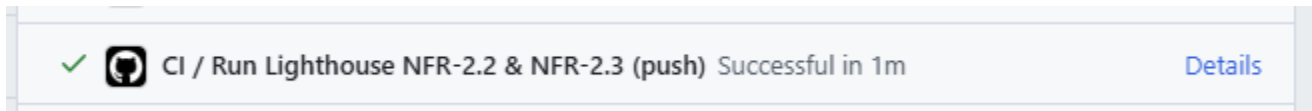
NFR-2.2

This test was conducted in the pipelines, and when it passed, a screenshot is displayed below.



NFR-2.3

This test was conducted in the pipelines, and when it passed, a screenshot is displayed below.



NFR-2.4

```
Local price hit: CSP500.JO  
Local price hit: CTOP50.JO  
Local price hit: EASYAI.JO  
Local price hit: STXEMG.JO
```

```
Fundamentals cache hit: MTN  
Fundamentals cache hit: NPN.JO  
Fundamentals cache hit: CSP500.JO  
Fundamentals cache hit: CTOP50.JO  
Fundamentals cache hit: EASYAI.JO  
Fundamentals cache hit: STXEMG.JO
```

So, here are the backend logs, the one on the left is shown when the user clicked the analytics page for the first time and the second image on the left is when the user presses the analytics page (for the second time) it say's "cache hit" in the backend logs.

Reliability Testing

These tests were run through the pipelines(testing) and this screenshot taken from there.

ID	Executable tests/ procedures	Tool	Target/ expected results	Actual result/ evidence
NFR-3.1	Make the Market, News, or AI service fail, then see if the other aspects are still functional.	Pytest	Except for the feature that failed, the remaining features should still be functional.	See evidence below
NFR-3.2	Make the AI fail and see if the API can manage it.	Pytest	The API does not crash the system and provides an accurate response.	See evidence below
NFR-3.3	Make the market data fail for one or more stocks in the system.	Pytest	The other stocks should still appear without the failing one.	See evidence below

Evidence Reliability Testing

These images are taken from the pipelines, but you can run them from the terminal, using the following command shown below.

NFR-3.1

Command to run:

- `cd backend`
- `python -m pytest tests/integration/test_nfr_failure_isolation.py -v`

```
33 tests/integration/test_nfr_failure_isolation.py . [ 6%]
```

NFR-3.2

Command to run:

- `cd backend`
- `python -m pytest tests/integration/test_nfr_ai_failure.py -v`

```
32 tests/integration/test_nfr_ai_failure.py . [ 5%]
```

NFR-3.3

Command to run:

- `cd backend`
- `python -m pytest tests/integration/test_nfr_holding_failure.py -v`

Scalability Testing

All these tests were done locally, and the results are shown below with a photo

ID	Executable tests/ procedures	Tool	Target/ expected results	Actual result/ evidence
NFR-4.1	Test the system simultaneously with 50 users.	K6	Here, we must ensure that 99% of all requests are successful and that 95% of them respond in less than two seconds.	See evidence below
NFR-4.2	Send 100 simultaneous API Requests to EquityLens	K6	At least 99% of requests succeed	See evidence below
NFR-4.3	To compare their performances while increasing the workload by 200%.	K6	little significant system modifications are required, and performance drops by little more than 10%.	See evidence below

Evidence Scalability Testing

NFR-4.1

Command to run: k6 run backend/tests/K6/scalability_nfr_4.1.js

```
PS C:\Users\Abdul\Documents\Equity-Lens> k6 run backend/tests/K6/scalability_nfr_4.1.js

      A  N  K6
     / \ / \ / \
    /   /   /   \
   /___/___/___\

execution: local
  script: backend/tests/K6/scalability_nfr_4.1.js
  output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
 * default: 50 looping VUs for 30s (gracefulStop: 30s)

THRESHOLDS

checks
✓ 'rate==1.0' rate=100.00%

http_req_duration
✓ 'p(95)<2000' p(95)=233.15ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 14954 480.694174/s
checks_succeeded...: 100.00% 14954 out of 14954
checks_failed.....: 0.00% 0 out of 14954

✓ status is 200

HTTP
http_req_duration.....: avg=99ms min=24.1ms med=80.58ms max=725.45ms p(90)=162.69ms p(95)=233.15ms
{ expected_response:true }...: avg=99ms min=24.1ms med=80.58ms max=725.45ms p(90)=162.69ms p(95)=233.15ms
http_req_failed.....: 0.00% 0 out of 14954
http_reqs.....: 14954 480.694174/s

EXECUTION
iteration_duration.....: avg=101.71ms min=24.1ms med=81.29ms max=1.27s p(90)=166.72ms p(95)=237.04ms
iterations.....: 14954 480.694174/s
vus.....: 3 min=3 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 3.9 MB 124 kB/s
data_sent.....: 1.8 MB 59 kB/s

running (0m31.1s), 00/50 VUs, 14954 complete and 0 interrupted iterations
```

NFR-4.2

Command to run: `k6 run backend/tests/K6/scalability_nfr_4.2.js`

```
PS C:\Users\Abdul\Documents\Equity-Lens> k6 run backend/tests/K6/scalability_nfr_4.2.js
```



```
execution: local
  script: backend/tests/K6/scalability_nfr_4.2.js
  output: -
```

```
scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
    * default: 50 looping VUs for 30s (gracefulStop: 30s)
```

THRESHOLDS

```
checks
✓ 'rate==1.0' rate=100.00%
```

TOTAL RESULTS

```
checks_total.....: 9689    318.260783/s
checks_succeeded...: 100.00% 9689 out of 9689
checks_failed.....: 0.00%   0 out of 9689
```

✓ status is 200

HTTP

```

http_req_duration.....: avg=152.85ms min=39.89ms med=126.57ms max=766.68ms p(90)=268.01ms p(95)=365.72ms
  { expected_response:true }...: avg=152.85ms min=39.89ms med=126.57ms max=766.68ms p(90)=268.01ms p(95)=365.72ms
http_req_failed.....: 0.00% 0 out of 9689
http_reqs.....: 9689 318.260783/s

```

EXECUTION

```
iteration_duration.....: avg=155.11ms min=39.89ms med=127.33ms max=773.09ms p(90)=275.77ms p(95)=367.77ms
iterations.....: 9689      318.260783/s
vus.....: 50      min=50      max=50
vus_max.....: 50      min=50      max=50
```

NETWORK

```
data_received.....: 2.5 MB 82 kB/s
data_sent.....: 1.2 MB 39 kB/s
```

```
running (0m30.4s), 00/50 VUs, 9689 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
```

Command to run: `k6 run backend/tests/K6/scalability_nfr_4.3.js`

C:\Users\Abdul\Documents\Equity Lens\

Usability and Accessibility Testing

Some of these tests were done locally

ID	Executable tests/ procedures	Tool	Target/ expected results	Actual result/ evidence
NFR-5.1	We measure the time while the user registers, logs in, imports a PDF, and opens the dashboard.	Live system Testing	Complete everything within 5 minutes	See evidence below
NFR-5.2	In here we need to check the keyboard navigation and the focus and the text contrast	Live system tests + chrome DevTools	The keyboard works; the focus must be visible + constant ratio > 4.5:1	See evidence below
NFR-5.3	Every financial indicator in the system should have a simple explanation for the user.	Live system tests	Every indicator has a simple explanation	See evidence below
NFR-5.4	The website must be accessible on a desktop, laptop, and phone.	Playwright	A website that doesn't require horizontal scrolling can function on many screens and mobile devices.	See evidence below

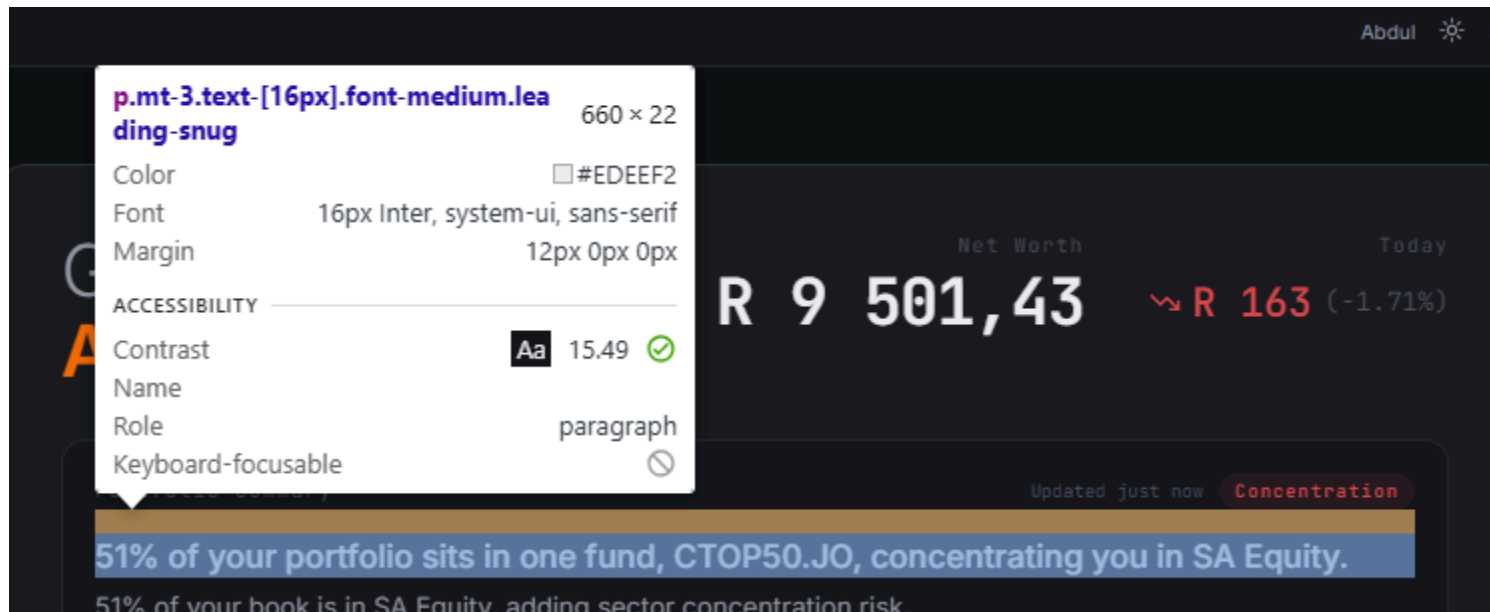
Evidence Usability and Accessibility Testing

NFR-5.1



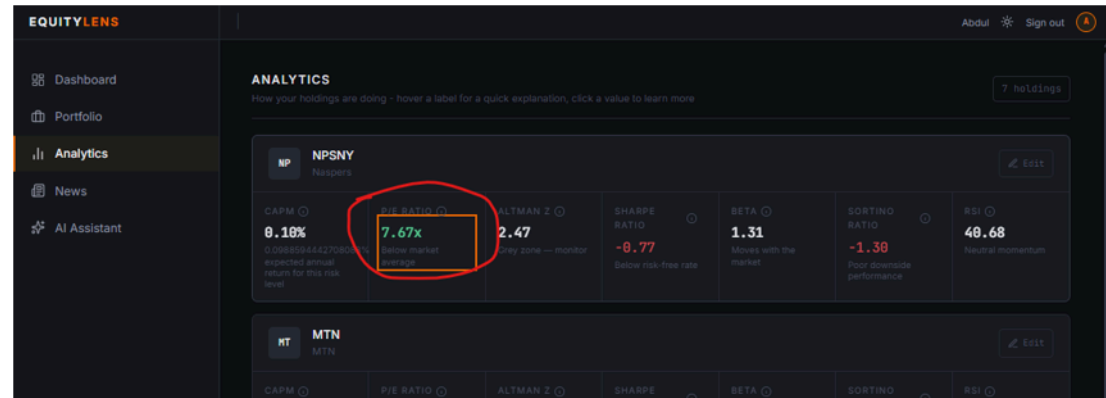
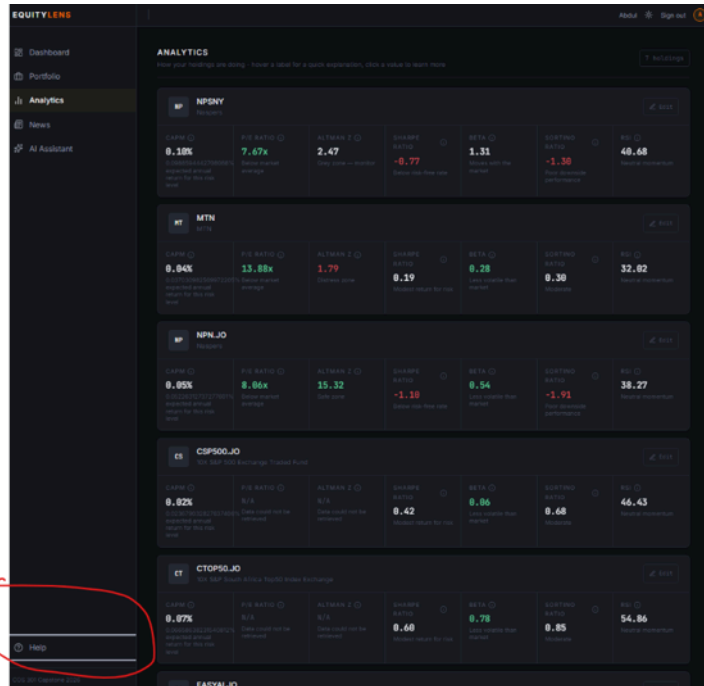
So, in here we did a timer and tested how long it takes for the user to register, then log in, then import the pdf, then view the dashboard as seen above from the timer, it was less than 5 minutes and there was still 1 minutes and 28 seconds left.

NFR-5.2

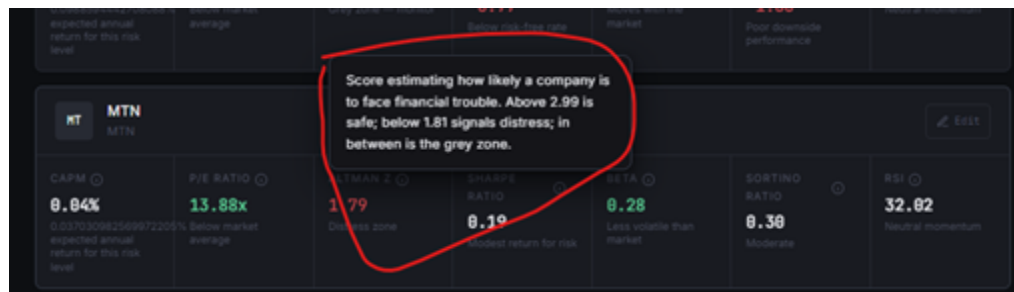
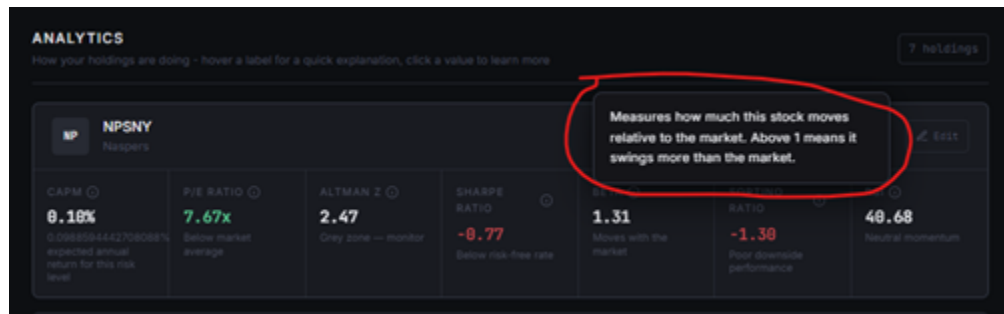


In here we can see that it meets our requirements (constant ratio > 4.5:1) - for the above image

In here we can see that when pressing the tab, it moves from one place to another, this meets our requirement for 5.2



NFR-5.3



In the images above, we can see that if you don't understand any of this calculation you can hover on any of them, then an explanation will be given.

NFR-5.4

This test was run locally and as seen below it has passed

Command to run: `npx playwright test test/nfr/nfr_5_4.spec.js`

```
PS C:\Users\Abdul\Documents\Equity-Lens\frontend> npx playwright test test/nfr/nfr_5_4.spec.js

Running 32 tests using 1 worker
  32 passed (2.9m)

To open last HTML report run:

  npx playwright show-report
```

Executable and Repeatable NFR Tests

ID	Quantified requirement	Tactics in SAS	Test/Tool	Target/actual
NFR-1.1	100% of API endpoints, other than the health check and the registration/login/verification endpoints, shall reject unauthenticated requests with an HTTP 401 response.	On the protected routes, this will serve as the primary authorization and token validation system.	OWSAP ZAP	100% 401 / 100% 401
NFR-1.2	User credentials shall never be stored in plain text by the system. Sessions shall be established with expiring access tokens with a lifetime of at most 60 minutes; an expired session shall require re-authentication or a silent token refresh before any protected resource is served.	This makes use of both the expired access token + the identity provider.	Live system test + AWS Cognito	0 plaintext text; <= 60 minutes / 0 plaintext text; <= 60 minutes
NFR-1.3	Multi-factor authentication (TOTP) shall be enforced on user accounts.	This is the second-factor authentication for TOTP.	Live Login Test	Valid TOTP / Valid TOTP
NFR-1.4	All data in transit shall be encrypted with TLS;	This guaranties both the encryption of the production data + HTTPS/TLS termination.	OpenSSL	TLS / TLS
NFR 1.5	All portfolio data shall be scoped to the owning user at the data-access level: 0	User ID checks of all the users.	OWSAP	0 cross-user access / 0 cross-user access

	instances of cross-user data exposure.			
NFR-1.6	All portfolio data shall be scoped to the owning user at the data-access level: 0 instances of cross-user data exposure, verified by integration tests.	Owner-scoped authorization + data access checks	GitLeaks	0 exposure / 0 exposure
NFR-1.7	All personal data handling shall comply with POPIA.	Data minimization + access control	OWSAP ZAP	All defined controls passed for the POPIA
NFR-2.1	API endpoints shall respond within 500 ms at the 85th percentile under normal load.	Fast API + database processing	K6	<= 500 ms / <= 500 ms
NFR-2.2	Initial page load shall complete within 10 seconds on a standard broadband connection.	Fast frontend + assets loading	Lighthouse	<= 10 s / <= 10 s
NFR-2.3	Dashboard visualisations shall render within 6 seconds	Fast dashboard data loading	Lighthouse	<= 6 s / <= 6 s

NFR-2.4	Market data shall be served through a cache-aside store: a request for an instrument that is already cached shall be answered with zero calls to the upstream provider, keeping repeat dashboard loads within provider rate limits	Cache market data to avoid extra API calls	Live API Test	0 upstream calls / 0 upstream calls
NFR-3.1	The failure of any single external data source (market data, news, AI model) shall degrade only its own feature; the remainder of the system shall stay fully functional.	The failure isolation + graceful degradation	Pytest	Other features in the system remain functional
NFR-3.2	Show a valid error message if the AI feature is not working	Error handling for the ai service	Pytest	Valid error shown / Valid error shown /
NFR-3.3	A failed fetch for one holding's market data shall not prevent the dashboard from rendering the remaining holdings.	Error isolation	Pytest	The remaining holders rendering
NFR-4.1	The backend shall support at least 50 concurrent users without performance degradation beyond the NFR-2 targets.	Concurrent User Handling	K6	50 concurrent users/ 100% passed

NFR-4.2	Send 100 simultaneous API Requests to EquityLens	High Request Handling	K6	100 simultaneous request/ 100% passed
NFR-4.3	The system shall accommodate a 200% workload increase without major architectural changes and with no more than a 10% performance decrease.	Workload Scaling Support	K6	Test passed after increased the workload by 200% / Test passed after increased the workload by 200%
NFR-5.1	A new user shall be able to register, import a portfolio and reach a populated dashboard within 5 minutes of first use.	Clear validation feedback for the user + easy user setup	Live system Testing	<= 5 minutes / <= 5 minutes
NFR-5.2	The interface shall meet WCAG 2.1 AA: all interactive elements keyboard-navigable with visible focus states, and text contrast ratios of at least 4.5:1.	Accessible UI Design	Live system tests + chrome DevTools	Keyboard + visible focus + >= 4.5:1 > Paased + >= 4.5:1

NFR-5.3	Every financial indicator shall have a plain-language explanation accessible from where the indicator is displayed, so no finance training is required to use the system.	Explanations of each indicator + Contextual help	Live system tests	100% indicators / 100% indicators
NFR-5.4	The interface shall be fully responsive across desktop (1280px+), tablet (768-1279px) and mobile (320-767px) with no horizontal scrolling on mobile.	Responsive layout + it can be used in mobile phones + flexible grid design	Playwright	0 horizontal scroll / 0 horizontal scroll

